

MELIKA

Sistema de Gestión de Historias Clínicas

Análisis DOFA

El presente análisis DOFA evalúa el estado actual del proyecto Melika desde una perspectiva técnica y estratégica, identificando los factores internos (fortalezas y debilidades) y externos (oportunidades y amenazas) que inciden en su desarrollo, estabilidad y proyección como solución de software para la gestión de historias clínicas. El diagnóstico toma como base la arquitectura implementada — React y Vite en el frontend, Node.js con Express y PostgreSQL en el backend, despliegue en Railway y manejo de documentos clínicos en PDF — así como los hallazgos de la auditoría de módulos realizada recientemente sobre el sistema.

FORTALEZAS (Internas · Positivas)	DEBILIDADES (Internas · Negativas)
- Arquitectura desacoplada cliente-servidor: SPA en React + Vite consumiendo una API REST en Express, lo que permite escalar y desplegar cada capa de forma independiente. - Persistencia en PostgreSQL con acceso mediante pool de conexiones, optimizando el manejo concurrente de consultas en operaciones críticas sobre historias clínicas. - Generación nativa de documentos clínicos en PDF con @react-pdf/renderer, permitiendo exportar historias, fórmulas y aclaraciones directamente desde el navegador. - Visualización de PDF embebida mediante @pdfslick/react (v4), brindando una experiencia de lectura fluida a pacientes y médicos sin salir de la plataforma. - Separación clara de roles funcionales (paciente / médico), con flujos de trabajo diferenciados que reflejan procesos clínicos reales. - Despliegue unificado en Railway (frontend, backend y base de datos), simplificando la infraestructura y reduciendo el tiempo de puesta en producción. - Cultura de auditoría de código ya instaurada: se realizó una revisión integral del proyecto que permitió detectar y documentar errores concretos antes de llegar a producción. - Stack liviano y moderno (JavaScript y CSS puro), sin dependencias pesadas de frameworks visuales, lo que reduce el peso del bundle y mejora los tiempos de carga.	- Bug crítico en el flujo de cancelación de citas: la ruta asociada está rota, impidiendo completar una operación esencial para el paciente. - El componente FormularioAclaracion nunca se monta en el árbol de React, dejando una funcionalidad completa inaccesible pese a estar implementada. - El evento personalizado melika:aclaracion-creada se dispara pero no tiene listener asociado, por lo que la interfaz no se refresca tras crear una aclaración. - Ausencia evidente de una suite de pruebas automatizadas (unitarias o E2E) que hubiera permitido detectar estos bugs antes de la auditoría manual. - CSS puro sin un sistema de diseño ni librería de componentes, lo que puede generar inconsistencias visuales y mayor esfuerzo de mantenimiento a futuro. - Doble dependencia en el manejo de PDF (@react-pdf/renderer para generación y @pdfslick/react para visualización) que exige mantener ambas versiones sincronizadas. - Falta de manejo centralizado de logs y errores en el backend, lo que dificulta el diagnóstico ágil de incidentes en producción.
OPORTUNIDADES (Externas · Positivas)	AMENAZAS (Externas · Negativas)

- Alta demanda de digitalización en el sector salud colombiano: numerosas IPS y consultorios pequeños aún gestionan historias clínicas en papel o Excel. - Marco normativo favorable en Colombia hacia la Historia Clínica Electrónica, que impulsa la adopción de sistemas como Melika por parte de entidades de salud. - Posibilidad de incorporar firma electrónica en los PDF ya generados, aportando validez legal a historias y fórmulas médicas. - Expansión hacia telemedicina: la arquitectura API REST facilita sumar videoconsulta o mensajería médico-paciente sin rediseñar el backend. - Interoperabilidad futura con estándares como HL7/FHIR, abriendo la puerta a integraciones con EPS, laboratorios y farmacias. - Escalabilidad natural hacia una aplicación móvil (React Native) reutilizando la misma API REST ya construida. - Migración de PDFs y evidencias clínicas a almacenamiento en la nube (S3, Cloudinary), liberando carga del servidor en Railway.	- Alta sensibilidad de los datos gestionados: al tratarse de historias clínicas, el proyecto debe cumplir estrictamente la Ley 1581 de 2012 de protección de datos personales; cualquier brecha tiene implicaciones legales serias. - Competencia de soluciones ya consolidadas en el mercado de software médico, con mayor trayectoria, certificaciones y base de usuarios. - Dependencia de un único proveedor de infraestructura (Railway) para frontend, backend y base de datos, concentrando el riesgo ante caídas o cambios de política del servicio. - Riesgo reputacional si los bugs ya identificados (cancelación de citas, aclaraciones) llegan a producción sin corregirse antes del lanzamiento. - Vulnerabilidades potenciales en dependencias npm si no se auditan y actualizan periódicamente, en especial en librerías de PDF y autenticación. - En software del sector salud, un error visible para el usuario puede erosionar rápidamente la confianza de médicos y pacientes en la plataforma.
--	--

Estrategias derivadas del análisis (matriz TOWS)

A partir del cruce de los cuatro componentes del DOFA se plantean las siguientes líneas de acción, priorizadas según su impacto en la estabilidad, seguridad y crecimiento del proyecto.

FO — Fortalezas + Oportunidades	Aprovechar el módulo de generación de PDF ya construido para incorporar firma electrónica y cumplir la normativa de Historia Clínica Electrónica, diferenciando a Melika frente a la competencia. Usar la arquitectura API REST como base para explorar integraciones HL7/FHIR con EPS y laboratorios.
FA — Fortalezas + Amenazas	Priorizar la corrección de los tres bugs detectados (ruta de cancelación de citas, montaje de FormularioAclaracion y listener del evento de aclaración) antes de exponer el sistema a usuarios reales, protegiendo la reputación del producto. Reforzar el cifrado y los controles de acceso alineados a la Ley 1581 para blindar los datos clínicos sensibles.
DO — Debilidades + Oportunidades	Construir una suite de pruebas automatizadas (Jest para backend, Playwright para flujos E2E) que permita detectar bugs como los ya identificados antes de que lleguen a producción. Aprovechar la creciente demanda de digitalización en salud como motivación para invertir en un sistema de diseño CSS consistente.
DA — Debilidades + Amenazas	Migrar el almacenamiento de los PDF generados a un proveedor externo (S3 o Cloudinary) para reducir la dependencia única de Railway. Establecer un pipeline recurrente de auditoría de dependencias (npm audit) y de manejo centralizado de logs para mitigar vulnerabilidades y agilizar el diagnóstico de incidentes.

Conclusión

Melika parte de una base técnica sólida — arquitectura desacoplada, manejo transaccional en PostgreSQL y generación/visualización de PDF ya integradas — que la posicionan favorablemente frente a la creciente demanda de digitalización en el sector salud. Sin embargo, dado que el sistema maneja información clínica sensible, la prioridad inmediata debe ser doble: corregir los bugs funcionales ya identificados en la auditoría y fortalecer los controles de seguridad y cumplimiento normativo, antes de avanzar hacia las oportunidades de expansión (firma electrónica, interoperabilidad, aplicación móvil).